Exactly n Times — For Loops

AP CSP · Unit 3, Lesson 2 · Reference Sheet
Read page 1 before class

When you know **exactly how many**, a `for` loop puts all the counting on one line: start, question and update. The body is the only thing left. Keep this beside you while you build; leave page 2 face-down until after.

Three jobs, one line

FOR LOOP

```
for (let i = 0; i < 200; i = i + 1) {
  circle(random(width), random(height), 2);
}
```

PART	ITS JOB
`let i = 0`	**Start.** Runs once, first.
`i < 200`	**Question.** Before every trip.
`i = i + 1`	**Update.** After every trip.

SEMICOLONS
The three parts are separated by **semicolons**, not commas. `i++` is short for `i = i + 1`.

Trace it

COUNT FROM ZERO

```
let total = 0;
for (let i = 0; i < 3; i++) {
  total = total + 10;
}
```

I	I < 3?	TOTAL
0	true	10
1	true	20
2	true	30
3	false	stop

ORDER, EVERY TIME
start once → question → body → update → question … The update is written first but runs **last**.

Where you start, where you stop

HEADER	TRIPS
`let i = 0; i < 5`	**5**
`let i = 1; i <= 5`	**5**
`let i = 0; i <= 5`	**6** (one too many)
`let i = 1; i < 5`	**4** (one too few)

THE HABIT
Start at **0**, use **<**. Then the number after < **is** the number of trips.

while or for?

USE	WHEN
`for`	You know how many: 200 stars, `rocks` asteroids.
`while`	Until something happens: until it is far enough.

Either can do the other's job. Pick the one that says what you mean.

What this looks like on the AP exam

REFERENCE SHEET

```
REPEAT 200 TIMES
{
  circle(RANDOM(1, 800), RANDOM(1, 600), 2)
}
```

ON THE EXAM · AAP-2.K
`REPEAT n TIMES` has **no loop variable**. Translate it as `for (let i = 0; i < n; i++)`. If the pseudocode needs a counter, it makes its own: `d ← 60` … `d ← d + 30`.

Check yourself before class

1. `for (let i = 0; i < 8; i++)` — how many trips?
2. `for (let i = 3; i < 8; i++)` — how many trips?
3. Write the header for `REPEAT 25 TIMES`.
4. What is i the first time the body runs?
5. `for (let i = 0; i < 10; i = i + 5)` — trips?
6. Commas or semicolons between the three parts?

Answers. 1) 8. 2) 5 (3, 4, 5, 6, 7). 3) `for (let i = 0; i < 25; i++)`. 4) 0. 5) 2 (0, 5). 6) Semicolons.

PAGE 2 — after the build. The patterns behind TODOs 2–5 and the coins lab.

Rewriting a while as a for

TODO 3

```
// while: the count takes three lines
let count = 0;
while (count < lives) {
  triangle(x, 18, x - 10, 42, x + 10, 42);
  x = x + 40;
  count = count + 1;
}

// for: the same three, on one line
for (let count = 0; count < lives; count++) {
  triangle(x, 18, x - 10, 42, x + 10, 42);
  x = x + 40;
}
```

ONLY THE COUNTER MOVES
`x` is not the counter. It keeps its own start (before the loop) and update (inside).

n can be a variable

TODO 2 + 4

```
for (let i = 0; i < rocks; i++) {
  circle(random(width), random(380), 50);
}
```

Type `rocks`, not `6`. When a key changes `rocks`, the loop runs a different number of times on the very next frame. If `rocks` is 0, the body runs **zero** times — nothing draws and there is no error.

A counter inside REPEAT n TIMES

TODO 5

```
let d = 60;              // d ← 60, BEFORE
for (let i = 0; i < 3; i++) {
  circle(SHIP_X, SHIP_Y, d);
  d = d + 30;             // d ← d + 30, INSIDE
}
```

WHERE THE START GOES
`let d = 60` inside the loop resets d every trip: three rings drawn on top of each other.

i does not have to go up by 1

HEADER	I TAKES
`i = 0; i < 10; i = i + 2`	0, 2, 4, 6, 8
`i = 5; i > 0; i--`	5, 4, 3, 2, 1

Tomorrow you use the value of `i` itself, not just the number of trips.

Why the coins stay put

`randomSeed(5)` replays the same list of "random" numbers every frame. Adding a coin just uses the next numbers in the list, so the old ones land where they always did.

Three mistakes worth knowing about in advance

WRITTEN LIKE THIS	WHAT ACTUALLY HAPPENS
`for (let i = 0; i < 5; i++);`	The semicolon ends the loop. The braces below run **once**. No error.
`for (let i = 0, i < 5, i++)`	Commas instead of semicolons. Syntax error.
`for (let i = 0; i < 5; i--)`	i goes 0, −1, −2… always less than 5. Infinite loop, frozen tab.

AP CSP 2026–27 · Unit 3 Lesson 2 · 3.8 Iteration (AAP-2.J, AAP-2.K) · Skills 2.B Implement and apply an algorithm · 4.B Determine the result of code segments